VITYARTHI Project Report

Warehouse Robot Pathfinder

Grid based robot navigation using BFS algorithm in C++

Name :- Monil Gautam
Registration Number :- 25BCE10310
Branch :- CS (core)
Subject :- Fundamentals of AI/ML (CSA 2001)
Professor :- Mrs. Garima Jain
College :- VIT Bhopal

1. Problem Statement

In large warehouses operated by companies like Amazon and Flipkart, autonomous robots are responsible for picking up items from shelves and delivering them to a drop-off point. The warehouse floor is essentially a grid where some cells are blocked by shelves and the robot has to find a way through without hitting any of them.

The problem we are solving is: given a 10x10 grid with a start position, an end position, and some obstacle cells, find the shortest possible path from start to end such that no obstacle is crossed. The robot can only move in four directions which are up, down, left and right.

This is a standard pathfinding problem and is directly relevant to the course topics of graph traversal algorithms and data structures like queues and arrays.

2. Why This Problem Matters

Pathfinding is one of the most commonly applied problems in computer science. Amazon alone operates over 750,000 warehouse robots that solve this exact kind of problem continuously. The decisions made in path planning directly affect how fast orders get fulfilled and how efficiently the robots use the space.

From a course perspective this problem ties together multiple concepts we studied. It requires graph traversal through BFS, use of queues as the core data structure, 2D arrays for grid representation, and OOP design to structure the code properly. Solving it gives a clear picture of how these concepts work together in a real application rather than just in theory.

We chose this problem specifically because it has a visible output that is easy to understand and demonstrate, and because the real world connection to warehouse robotics makes it easy to explain the relevance.

3. Approach to Solving the Problem

3.1 Breaking it down

Before writing any code we broke the problem into smaller parts. First we needed a grid. Then we needed a way to check which cells a robot can move to. Then we needed the algorithm to find the path. Then we needed to display the result. Each of these became a separate class or function in the code.

3.2 Algorithm chosen — BFS

We chose Breadth First Search because it guarantees the shortest path in a grid where all moves have equal cost. BFS explores cells level by level starting from the closest ones so the first time it reaches the destination it has found the minimum number of steps to get there. No other cell at that distance has been skipped.

We also considered DFS but DFS does not guarantee the shortest path. It might find a path but it could be longer than necessary. For a warehouse robot, taking the shortest path matters because it directly affects delivery time.

3.3 Path reconstruction

One challenge with BFS is that the queue only tells you which cells were visited, not the actual path. To solve this we used two 2D arrays called parentRow and parentCol. Every time BFS visits a new cell it records which cell it came from. At the end we trace backwards from E to S using these arrays and mark every cell on the path with a * symbol.

3.4 Code structure

The code is divided into three classes. Grid handles the warehouse map. Robot stores position and reads neighboring cells. BFS runs the algorithm and does the traceback. This separation made the code easier to write, debug and explain.

Class	Responsibility
Grid	Creates and displays the 10x10 warehouse array
Robot	Stores robot position, holds reference to grid, reads neighbors
BFS	Runs BFS traversal, fills parent arrays, traces path back

4. Key Decisions Made

4.1 Array over vector for the grid

We decided to use a plain 2D character array instead of a vector of vectors for the grid. Since the grid size is fixed at 10x10 and never changes at runtime, a vector would add complexity with no benefit. Arrays are simpler to declare and access for a fixed size problem like this.

4.2 Separate setupGrid and displayGrid functions

Initially the grid setup and display were in the same function. This caused problems because BFS needs to read the grid values but calling the display function again would reset things. Splitting them into two separate functions fixed this and also made the code cleaner.

4.3 Robot holding a reference to Grid instead of inheriting it

At first Robot was inheriting from Grid which caused design problems. A robot is not a grid, it exists inside a grid. So we changed it to hold a reference to the Grid object instead. This is composition which is more accurate to the real relationship between these two things.

4.4 Checking destination after pop not before

We placed the destination check right after popping from the queue and reading the current cell. This is important because if you check before reading the neighbors you might miss recording the parent of the last cell. Getting this order right was important for the traceback to work correctly.

5. Challenges Faced

5.1 Understanding path traceback

The hardest part of the whole project was understanding how to get the path out of BFS after it finishes. BFS just explores the grid, it does not automatically give you the path. It took some time to understand that the parent arrays are what solve this and that the traceback is a completely separate step that happens after BFS is done.

5.2 Pass by value bug

One bug that took time to find was that the grid was being passed by value into functions. This meant the function was working on a copy of the grid and any changes made inside were not reflecting in the original object. The fix was to either not pass the object at all since the function was already inside the class, or to pass by reference.

5.3 Parent array direction

There was confusion about which cell's slot to store the parent in. Initially the code was storing the parent coordinates in the current cell's slot instead of the neighbor's slot. This means during traceback you would get wrong values. The correct approach is to always store in the neighbor's slot since you are recording how that neighbor was reached.

5.4 Queue and pair syntax

The syntax for queue with pair was new. Understanding that .first gives the row and .second gives the column, and that you have to read these before calling pop, took some trial and error to get right.

6. Code Organization and Documentation

The code is organized into three classes that each handle one part of the system. Comments are added above each class and key sections to explain what they do. The framework is also written as a comment block at the top of the file so anyone reading it can understand the overall structure before going into the details.

Section	Description
Grid class	Handles array declaration, grid setup with obstacles, and terminal display
Robot class	Stores position using map_row and map_col, holds Grid reference, reads neighbors
BFS class	Contains visited array, parent arrays, queue, pathFind() and tracePath() functions

main()	Creates objects, calls setup and display, runs BFS, displays result

Variable names are kept descriptive. parentRow and parentCol clearly explain what they store. visited is self explanatory. The grid symbols S, E, X, dot and star are consistent throughout the code and output.

OUTPUT (screenshot)

```
///////////////// Ware house path-finder


 ====================================

A robot in storage facility is going to find a way out of this maze
This is hardcoded map but it can be customized also according to the layout of the building




 Starting process......



Map layout


| S  .  X  .  .  .  .  .  .  . |
| .  .  X  .  .  .  .  .  .  . |
| .  .  X  .  .  .  .  .  X  . |
| .  .  X  .  .  .  .  .  X  . |
| .  .  .  .  .  .  .  .  X  . |
| X  X  X  X  X  X  .  X  X  X |
| .  .  .  .  .  .  .  .  .  . |
| .  X  X  .  .  .  X  .  .  . |
| .  X  .  .  .  .  X  X  .  . |
| .  .  .  .  .  .  .  .  .  E |



===========================================


Finding path...

| S  .  X  .  .  .  .  .  .  . |
| *  .  X  .  .  .  .  .  .  . |
| *  .  X  .  .  .  .  .  X  . |
| *  .  X  .  .  .  .  .  X  . |
| *  *  *  *  *  *  *  .  X  . |
| X  X  X  X  X  X  *  X  X  X |
| .  .  .  .  .  .  *  *  .  . |
| .  X  X  .  .  .  X  *  *  . |
| .  X  .  .  .  .  X  X  *  . |
| .  .  .  .  .  .  .  .  *  E |



Path found sucessfully !!

Terminating process.........

Thank You !
```

7. What I Learned

7.1 How BFS actually works in practice

Reading about BFS in theory and actually implementing it are very different things. Building it from scratch made it clear how the queue drives the exploration and why FIFO order is what guarantees the shortest path. The level by level expansion finally made sense when I traced through the grid manually.

7.2 Why design matters before coding

Early in the project the Robot class was inheriting Grid which seemed fine at first but caused problems later. Having to go back and fix the design after already writing code around it showed why spending time on design upfront saves a lot of time later. Thinking about whether Robot IS a Grid versus whether Robot USES a Grid would have saved that whole rework.

7.3 References in C++

The reference member in the Robot class was something new. Understanding that a reference is not a copy but an alias to the original object, and that it must be initialized in the constructor initializer list and not inside the function body, was a practical lesson that reading alone would not have made as clear.

7.4 Debugging with small examples

When the path traceback was giving wrong results, the way to find the bug was to trace through a small 3x3 example manually on paper and compare each step to what the code was doing. This approach of reducing the problem to a small case made bugs much easier to find than trying to debug on the full 10x10 grid.

8. Result

The BFS algorithm successfully finds and displays the shortest path from S to E on all tested map configurations. The path is marked with * symbols on the terminal grid. On more complex maps with a narrow opening the algorithm correctly identifies the only passage and routes through it.

The output clearly shows the grid before and after BFS runs, making it easy to visually verify that the path avoids all obstacles and takes the shortest route.

9. Future Scope

- Implement A* algorithm which uses a heuristic to search smarter and faster than BFS

- Add a map randomizer that generates a new valid solvable map every time the program runs
- Show the robot moving step by step on the terminal instead of just showing the final path
- Support loading warehouse layouts from external text files for more flexibility
- Extend to multiple robots on the same grid that avoid each other

10. Conclusion

This project was a good exercise in applying course concepts to a real world problem. BFS, queues, 2D arrays and OOP design all came together in one working system which is something that individual assignments do not always show.

The warehouse robot scenario gave the problem a clear real world context. It was easier to make design decisions when thinking about what a real system would need rather than just writing code abstractly. The challenges faced, especially around path traceback and class design, gave more practical understanding than any theoretical explanation could.

If given more time the A* algorithm and a dynamic map generator would make the project significantly more complete. But the current implementation covers the core problem and demonstrates the key concepts clearly.